

Lekcja 08 - Production readiness w Pythonie

2026-08-14

Data Engineering Course

Gotowość produkcyjna w Pythonie

```
układ src -> CLI -> testy -> logowanie -> retry ->
runbook
```

To nie jest „większy framework”, tylko porządek. Porządek pozwala komuś innemu uruchomić pipeline, zrozumieć logi i naprawić go bez zgadywania.

Skrypt, który działa lokalnie, nie musi być gotowy do review. Minimalny standard produkcyjny wygląda tak:

- czytelny układ src
- jawny punkt wejścia (CLI)
- testy dla logiki
- sensowne logi
- ograniczone retry dla błędów przejściowych
- README + runbook dla operacji

Jeśli czegoś z tego nie ma, pipeline jest tylko demo.

■ Problem

Skrypt działa lokalnie, ale nie da się go bezpiecznie uruchomić w pracy. Zwykle pojawiają się pytania:

```
jak podałem dane wejściowe?  
gdzie zapisuje wynik?  
gdzie są logi?  
jak zrobić ponowne uruchomienie po błędzie?  
czy transformacja ma testy?
```

Jeśli brak odpowiedzi, to nie ma odpowiedzialności i nie ma zaufania do procesu.

■ Dlaczego to jest ważne

W pracy nie chodzi o to, czy kod jest „ładny” albo „czy działa na moim laptopie”. Chodzi o to, czy ktoś inny potrafi:

- uruchomić job
- zrozumieć wejście i wyjście
- odtworzyć problem
- sprawdzić, czy błędy są przejściowe czy biznesowe
- prowadzić działający pipeline bez tajemnic

To jest kluczowy poziom dojrzałości inżynierii danych.

■ Minimalny standard projektu

```
src/  
tests/  
README.md  
runbook.md  
CLI  
logging  
retry policy
```

Nie potrzebujesz ciężkiego frameworku. Potrzebujesz czytelnego minimum architektonicznego, które można sensownie utrzymywać.

```
src/pipeline/  
  cli.py  
  extract.py  
  transform.py  
  load.py  
  
tests/  
  test_transform.py  
  test_contracts.py
```

Moduły są małe, mają jedną odpowiedzialność i łatwiej je review'ować. To ważne nie tylko dla Pythona, ale dla całego data engineering.

■ Co oznacza dobrze zorganizowany moduł

```
extract.py -> pobiera dane i wykonuje parsowanie  
transform.py -> normalizuje i agreguje  
load.py -> zapisuje wynik na końcu  
models.py -> kontrakty i typy danych  
cli.py -> punkt wejścia i parsowanie argumentów
```

Dobrze podzielony moduł jest łatwiejszy do testowania, debugowania i wyjaśniania problemów bez zgadywania.

```
python -m pipeline.cli --input data/orders.json  
--output output/result.json
```

CLI robi dwie rzeczy: usuwa ręczne „edytowanie kodu pod konkretne dane” i daje jasno zdefiniowany punkt wejścia. To jest bardzo ważne przy pracy w zespole.

```
logger.info("Loaded records", extra={"count": len  
(records)})
```

Log powinien powiedzieć:

```
co startowało  
ile rekordów weszło  
co zapisano  
czy wystąpił błąd i na którym etapie
```

Dobrze napisany log jest pomostem między developerem a operatorem pipeline.

■ Logowanie kontra debugowanie

debugging: pokazuje szczegóły techniczne
logging: pokazuje stan operacyjny

Dobrze ustawiony log nie musi być ogromny. Ma pokazać „co się dzieło” w sensie operacji, nie tylko „stack trace w środku nocy”.

```
błąd przejściowy I/O / timeout -> retry ma sens  
niepoprawny payload -> retry nie ma sensu
```

Błąd z sieci to inna klasa problemu niż błąd danych. Retry nie jest rozwiązaniem na wszystko.

■ Polityka retry: ograniczona i sensowna

- max 3-5 prób
- exponential backoff
- tylko dla błędów typu 429 / timeout / 5xx
- nie retry dla 400 / 401 / błędu kontraktu

To jest ważne, bo retry „na siłę” zamienia problem jakości danych w zjawisko operacyjne i zwiększa koszty.

```
def test_completed_revenue_counts_only_completed()
-> None:
    records = [
        {"status": "completed", "total_amount":
100.0},
        {"status": "pending", "total_amount": 50.0
    },
    ]
    assert calculate_completed_revenue(records) ==
100.0
```

Testy powinny sprawdzać czystą logikę bez realnego schedulera, AWS i zewnętrznych zależności. To daje zaufanie do transformacji.

■ Jaki poziom testów ma sens

- unit tests dla funkcji transformacji
- testy dla reguł walidacji
- małe golden tests dla znanych danych wejściowych
- brak testów dla zewnętrznego systemu w środowisku produkcyjnym

Celem nie jest „testować wszystkiego”, tylko mieć zaufane, małe fragmenty logiki, które można szybko sprawdzić.

Runbook to instrukcja po awarii. Powinien zawierać:

```
jak uruchomić job  
jak wykonać rerun po błędzie  
gdzie są wejścia i wyjścia  
gdzie są logi  
najczęstsze błędy  
właściciel danych / kontakt do źródła
```

Dobre runbooki są zwykle bardziej cenione niż błyskotliwe demo.

Definition of Done

```
[ ] projekt ma czytelny układ src  
[ ] pipeline ma CLI  
[ ] logi są sensowne i operacyjne  
[ ] retry jest ograniczony do błędów przejściowych  
[ ] testy obejmują transformacje i kontrakty  
[ ] README i runbook mówią, jak uruchomić i  
naprawić projekt
```

To jest pierwszy krok od „demo” do „prawie produkcyjnego projektu”.

Pytanie:

Jak zbudować mały produkcyjny projekt ETL w Pythonie?

Odpowiedź:

Dzielę pobieranie danych, transformację i ładowanie na małe moduły w src.
Ekspozuję przepływ przez CLI z jasnymi argumentami wejściowymi i wyjściowymi.
Zachowuję czystą logikę pod testami i zapisuję logi dla punktów wejścia i błędów.
Błędy sieciowe mają ograniczone retry, a niepoprawne dane są odrzucane, a nie retryowane w nieskończoność.
README i runbook opisują, jak uruchomić i odzyskać job.

Checklist do powtórki

- ☐ Czy potrafisz opisać minimalny produkcyjny układ projektu?
- ☐ Czy rozumiesz, dlaczego CLI jest ważne?
- ☐ Czy umiesz odróżnić błąd przejściowy od błędu kontraktu?
- ☐ Czy wiesz, po co są runbook i README?
- ☐ Czy rozumiesz, że testy są częścią bezpieczeństwa operacyjnego, nie tylko „dobrego stylu”?

